

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3) Assessment

Tool Weightage: 10%

Duration :60 Mins

On Class

Total Marks:50

QUESTIONS: 1

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I Dinesh V (192524309) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dinesh V (192524309)

Annexure A

TECHNICAL PRESENTATION

Q1. Phases of a Compiler and Their Role

Understanding of Case

-
- a) Explains all compiler phases such as lexical, syntax, semantic, intermediate, optimization, and code generation.
 - b) Describes the role of each phase in transforming source code into machine code.

Application of Concepts

- c) Applies compiler phases to detect errors like invalid tokens, missing syntax, and semantic mismatches.
- d) Demonstrates phase-wise error identification using real C program examples.

Analysis

- e) Analyzes how each phase depends on the previous phase for correct execution.
- f) Evaluates how early-stage errors (lexical/syntax) stop further compilation.

Solution Design

- g) Suggests integrating better error recovery techniques across phases.
- h) Proposes modular compiler design for flexibility and easier debugging.

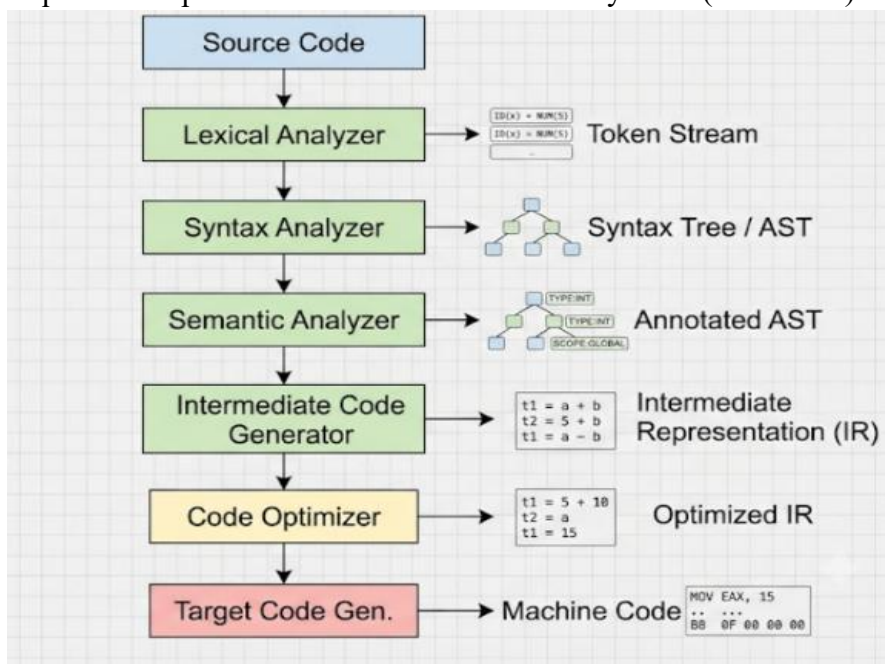
Clarity

- i) Uses flowcharts to explain phase sequence clearly.
- j) Provides step-by-step examples showing how code is processed.

Ans:

Understanding of Case & Role of Phases

A compiler translates high-level source code into executable target machine code through a series of sequential phases divided into Analysis (Front-End) and Synthesis (Back-End).



1. **Lexical Analysis (Scanner):** Reads raw character streams and groups them into meaningful units called *tokens*.
2. **Syntax Analysis (Parser):** Takes tokens and builds a Parse Tree or Abstract Syntax Tree (AST) to check grammatical structure.
3. **Semantic Analysis:** Checks for type consistency, scope resolution, and invalid operations (e.g., array index out of bounds, type mismatches).
4. **Intermediate Code Generation (ICG):** Converts the verified AST into a machine-independent code representation (e.g., Three-Address Code).
5. **Code Optimization:** Improves intermediate code to execute faster and consume less memory.
6. **Code Generation:** Converts optimized IR into target assembly or machine language instructions.

Application of Concepts & Phase-Wise Error Detection

Consider the following C code snippet containing errors across different phases:

C Program Code

```
int main () {
    int 1x = 10; // Error A
    float y = 5 + * 2; // Error B
    char* str = 100; // Error C
}
```

- **Lexical Phase:** Detects **1x** as an invalid identifier rule (identifiers cannot start with a digit).
- **Syntax Phase:** Detects **5 + * 2** as a syntax error due to two adjacent binary operators.
- **Semantic Phase:** Detects **char* str = 100** as a type mismatch error (assigning an int directly to a pointer without a cast).

Analysis: Inter-Phase Dependencies & Early Failure

- **Data Flow Dependency:** Each phase relies directly on the output structure generated by the preceding phase:

$$\text{Lexemes} \xrightarrow{\text{Lexer}} \text{Tokens} \xrightarrow{\text{Parser}} \text{AST} \xrightarrow{\text{Semantic}} \text{Annotated AST} \xrightarrow{\text{ICG}} \text{IR}$$

- **Cascading Error Suppression:** Early-stage errors (lexical or syntax) break the structural foundation required by subsequent phases. If the syntax analyzer encounters invalid syntax, it cannot construct a valid AST, making semantic verification and code generation impossible.

Solution Design

- **Panic-Mode & Phrase-Level Recovery:** Implement synchronization tokens (e.g., semicolons (;) or closing braces}) where the parser discards invalid tokens until a stable point is reached, allowing compilation to continue and catch further errors in a single pass.
- **Modular Compiler Design:** Decouple front-end analysis from back-end code generation using an intermediate representation (IR) such as LLVM IR. This allows adding new language front-ends or hardware back-ends independently.

Q2. Lexical Analysis and Token Specification using LEX

Understanding of Case

- a) Defines tokens, lexemes, and patterns used in lexical analysis.
- b) Explains the role of the lexical analyzer in scanning source code.

Application of Concepts

- c) Demonstrates token generation using LEX tool with sample inputs.
- d) Applies regular expressions to identify identifiers, constants, and operators.

Analysis

- e) Analyzes common lexical errors such as invalid identifiers and illegal symbols.
- f) Evaluates how tokenization improves parsing efficiency.

Solution Design

- g) Designs token rules using regular expressions in LEX.
- h) Implements error-handling mechanisms in lexical analysis.

Clarity

- i) Provides clear LEX syntax examples for better understanding.
- j) Uses input-output illustrations for token generation.

Ans:

Understanding of Case

- **Token:** An abstract category of language elements (e.g., KEYWORD, IDENTIFIER, NUMBER, ASSIGN_OP).
- **Lexeme:** The actual character sequence in the source code that matches a pattern for a token (e.g., int, totalSum, 45.2).
- **Pattern:** A formal rule expressed via regular expressions describing the set of strings forming a token.

Application of Concepts & LEX Implementation

Example Code

```
%{  
  
#include <stdio.h>  
  
%}  
  
DIGIT    [0-9]  
  
LETTER    [a-zA-Z_]  
  
ID        {LETTER}({LETTER}|{DIGIT})*  
  
NUM        {DIGIT}+(\.{DIGIT}+)?  
  
%%  
  
"if"|"else"|"while"|"int" {printf("TOKEN: KEYWORD (%s)\n", yytext); }  
  
{ID}                        { printf("TOKEN: IDENTIFIER (%s)\n", yytext); }  
  
{NUM}                       { printf("TOKEN: NUMBER (%s)\n", yytext); }  
  
"="                          { printf("TOKEN: ASSIGN_OP (%s)\n", yytext); }  
  
"+"|"-"|"*"|"/"             { printf("TOKEN: ARITH_OP (%s)\n", yytext); }  
  
[ \t\n]+                      ; /* Ignore whitespace */  
  
.                             { printf("LEXICAL ERROR: Unrecognized character '%s'\n", yytext); }  
  
%%
```

```
int main() {  
  
    yylex();  
  
    return 0;  
  
}  
  
int yywrap() { return 1; }
```

Input / Output Illustration

- **Input:** int total = 45;
- **Output:**
 - TOKEN: KEYWORD (int)
 - TOKEN: IDENTIFIER (total)
 - TOKEN: ASSIGN_OP (=)
 - TOKEN: NUMBER (45)

Analysis & Efficiency

- **Lexical Errors:** Occur when a character sequence cannot be matched against any defined regex pattern (e.g., 99variable without space handling, or unsupported characters like @, \$).
- **Parsing Efficiency:** Stripping white space, comments, and converting string streams into compact integer token IDs minimizes the memory overhead and complexity required by the syntax parser.

Solution Design

- **Regex Error Rule:** Catch-all rules placed at the end of the LEX rule section (.) intercept unexpected characters and output precise line-number error messages rather than halting execution abruptly.

Q3. Language Processors: Compiler vs Interpreter

Understanding of Case

- a) Differentiates between compiler and interpreter based on execution method.
- b) Explains how each processor converts high-level code.

Application of Concepts

- c) Applies concepts to languages like C (compiler) and Python (interpreter).
- d) Demonstrates how errors are handled differently in both.

Analysis

- e) Compares performance, speed, and memory usage of both processors.
- f) Evaluates advantages and disadvantages in real-world scenarios.

Solution Design

- g) Suggests using compilers for performance-critical applications.
- h) Recommends interpreters for rapid development and debugging.

Clarity

- i) Uses comparison tables for easy understanding.
- j) Includes real-life examples and execution flow diagrams.

Ans: Understanding of Case & Comparison

Feature	Compiler	Interpreter
Execution Method	Translates entire source code into machine code prior to execution.	Translates and executes source code line-by-line / instruction-by-instruction.
Execution Speed	Fast execution after initial compilation.	Slower execution due to runtime translation overhead.
Memory Usage	Higher initial memory footprint during build, low runtime footprint.	Keeps source code and interpreter environment loaded in memory.
Error Handling	Reports all lexical, syntax, and semantic errors together after scanning the full file.	Halts execution immediately upon encountering the first error at runtime.
Target Code	Produces independent target executable files (.exe, .o).	Does not produce persistent intermediate machine code files.
Example Languages	C, C++, Rust	Python, Ruby, JavaScript

Execution Flow Diagrams

Compiler Architecture

Source Code → Compiler → Machine Code (.exe) → Output

Interpreter Architecture

Source Code + Input Data → Interpreter → Output

Solution Design Recommendations

- **Compilers:** Ideal for performance-critical systems, system software, real-time control units, embedded applications, and game engines where execution speed and hardware resource management are primary concerns.
- **Interpreters:** Recommended for rapid prototyping, interactive data analysis, web scripts, and applications requiring quick debugging cycles without full re-compilation steps.

Q4. Grouping of Compiler Phases and Optimization

Understanding of Case

- a) Explains grouping into front-end (analysis) and back-end (synthesis).
- b) Describes the role of each group in compilation.

Application of Concepts

- c) Applies grouping to simplify compiler design and debugging.
- d) Demonstrates how front-end handles errors and back-end improves performance.

Analysis

- e) Analyzes how grouping improves modularity and maintainability.
- f) Evaluates optimization techniques like constant folding and dead code removal.

Solution Design

- g) Designs efficient compiler structure using grouped phases.
- h) Suggests advanced optimization techniques for better performance.

Clarity

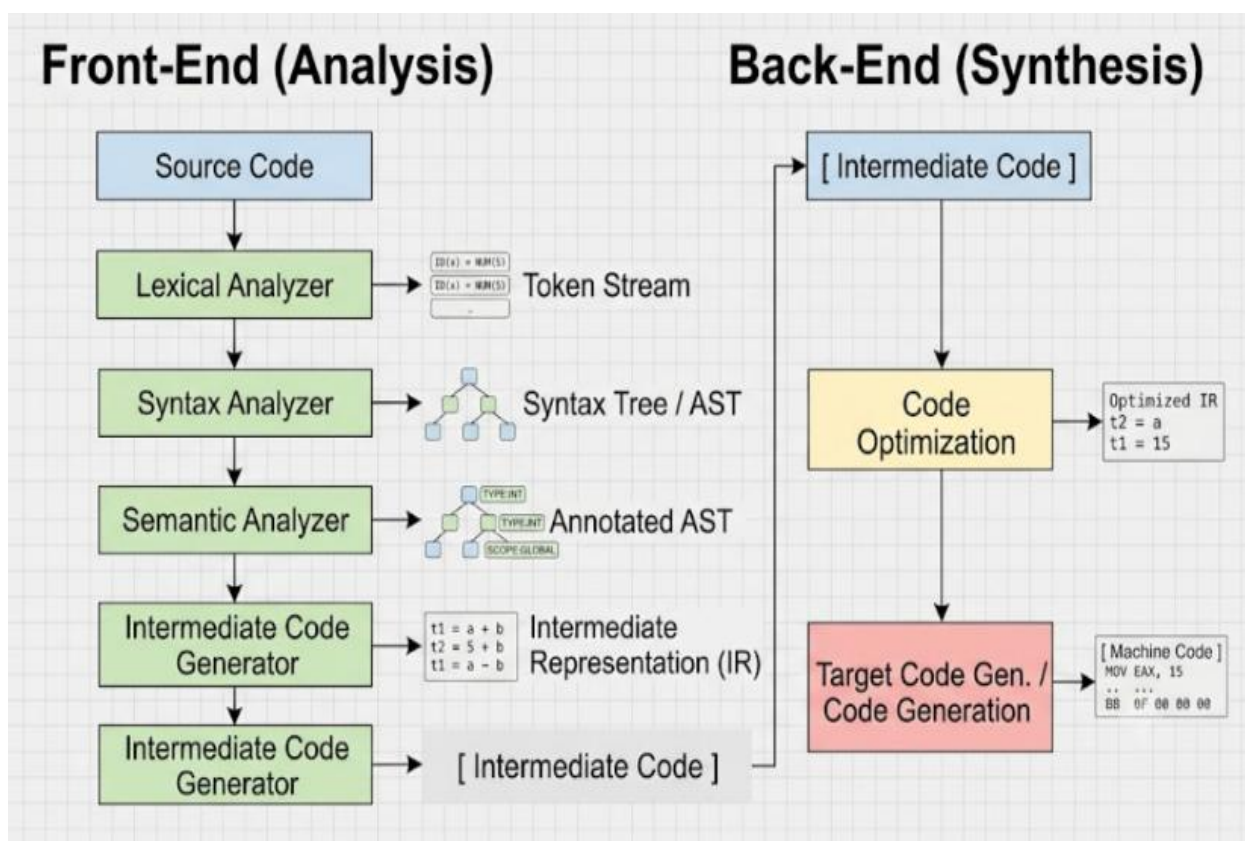
- i) Uses diagrams to represent front-end and back-end clearly.
- j) Provides before-and-after code examples for optimization.

Ans:

Understanding of Case & Grouping

Compiler phases are grouped into two primary modules:

1. **Front-End (Analysis Phase):** Language-dependent and machine-independent. It consists of Lexical Analysis, Syntax Analysis, Semantic Analysis, and Intermediate Code Generation.
2. **Back-End (Synthesis Phase):** Machine-dependent and language-independent. It consists of Code Optimization and Target Code Generation.



Analysis & Optimization Techniques

Modularity Benefits

Grouping separates source-language processing from target-architecture instructions. A single front-end for C can be paired with back-ends for x86, ARM, or RISC-V.

Optimization Examples

1. **Constant Folding:** Evaluates constant expressions at compile-time instead of runtime.

- *Before:* `int seconds = 60 * 60 * 24;`
- *After:* `int seconds = 86400;`

2. **Dead Code Elimination:** Removes code statements that do not affect program output or are unreachable.

- *Before:*

`int result = calculate();`

`return result;`

`int debugValue = 100; // Never reached`
- *After:*

`return calculate();`

Solution Design

- **Loop-Invariant Code Motion:** Move computations inside a loop that do not change across iterations outside the loop body to reduce overhead.
- **Peephole Optimization:** Inspect small sliding windows of target instruction sequences to replace redundant operations (e.g., replacing `MULT by 2` with a bitwise left-shift `SHL 1`).

Q5. Compiler Construction Tools and LEX Understanding of Concepts

- a) Explain compiler construction tools.
- b) Describe the structure of a LEX program.

Application of Concepts

- c) Apply LEX to generate tokens for a simple program.
- d) Demonstrate writing rules for identifiers and constants.

Analysis

- e) Analyze advantages of using LEX in compiler design.
- f) Evaluate limitations of LEX.

Solution / Design

- g) Design a simple lexical analyzer using LEX rules.

Clarity

- h) Show LEX workflow (input → scanner → tokens).

Ans:

Understanding Compiler Construction Tools

Compiler construction tools automate specific sub-tasks of compiler development:

- **LEX / FLEX:** Lexical Analyzer Generator (converts regular expressions into Finite Automata).
- **YACC / BISON:** Parser Generator (converts Context-Free Grammars into LALR parsers).
- **Syntax-Directed Translation Engines:** Generate intermediate code directly from parsed trees.
- **Code Generator Tools:** Produce targeted machine code from high-level operational rules.

Structure of a LEX Program

A standard.l file consists of three distinct sections separated by %%%:

```
%{
```

```
/* Section 1: Declarations */
```

```
/* C headers, macros, global variables */
```

```
%}
```

```
/* Definitions / Regex Aliases */
```

```
%%
```

```
/* Section 2: Rules */
```

```
/* [Pattern] { [C Action Code] } */
```

```
%%
```

```
/* Section 3: User Subroutines */
```

```
/* Main driver function and custom C helper functions */
```

LEX Specification File (.l) $\xrightarrow{\text{LEX Compiler}}$ lex.yy.c $\xrightarrow{\text{C Compiler (GCC)}}$ Executable Lexer $\xrightarrow{\text{Input Source Code}}$ Token Stream

Implementation: Lexical Analyzer Rules for Identifiers & Constants

```
%{
#include <stdio.h>

%}

%%

[a-zA-Z_][a-zA-Z0-9_]* { printf("Identifier: %s\n", yytext); }
[0-9]+(\.[0-9]+)?      { printf("Constant: %s\n", yytext); }
[ \t\n]                ; /* Skip spaces */
.                       { printf("Other character: %s\n", yytext); }

%%

int main() {
    yylex();
    return 0;
}

int yywrap() { return 1; }
```

Analysis: Advantages & Limitations

Advantages

- Automates complex Deterministic Finite Automata (DFA) state generation.
- Dramatically reduces development time and coding errors compared to manually writing hand-crafted scanners.
- Fast execution speeds for pattern matching.

Limitations

- Limited to regular languages (cannot handle nested structures, balancing parentheses, or context-free rules without pairing with a parser like YACC).
- Debugging state transition issues in large regex rule sets can be difficult.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-dept analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

